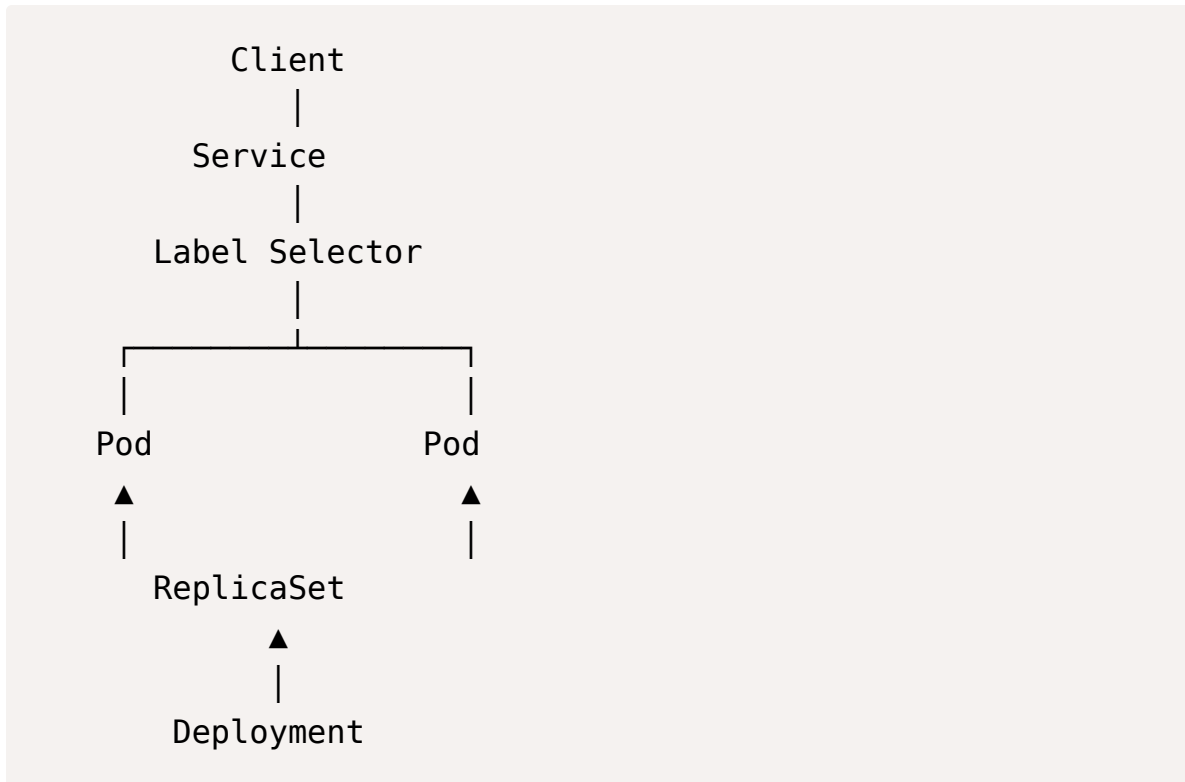


Deployment & Service

- Deployment는 ReplicaSet을 관리하고, ReplicaSet이 Pod를 관리함.
그리고 Deployment에서는 업그레이드 전략 등을 포함한 전반적인 어플리케이션 배포를 관리함.
Service는 생성된 Pod에 안정적으로 접근할 수 있도록 함.



- Pod
Kubernetes의 가장 작은 배포 단위로 보통 하나의 Pod에는 Application Container와 Sidecar Container(Optional)가 들어감.
※ Sidecar Container는 Pod안에서 메인 어플리케이션을 도와주는 보조 컨테이너임.
같은 Pod에 있는 컨테이너들은 네트워크, Volume(파일 시스템 일부), 생명주기를 공유함.
예) 로그 수집, 모니터링
- ReplicaSet
원하는 개수의 Pod를 유지하는 것으로 ReplicaSet은 주로 Deployment 안에서 관리됨.

- Service

Label로 선택된 Pod를 관리함.

※ DNS(Domain Name System)

종류	외부 접근	주 사용처
ClusterIP	✗	내부 마이크로서비스 간 통신(기본값)
NodePort	✓ (NodeIP:Port)	개발/테스트 환경, 간단한 외부 노출
LoadBalancer	✓ (클라우드 LB)	운영 환경에서 인터넷 서비스 제공
ExternalName	외부 DNS로 연결	외부 DB, 외부 API 등과 DNS 기반 연동

실무에서 가장 흔한 조합은 **Deployment + Service(ClusterIP)**이며, 외부에 서비스를 공개해야 할 때는 **Service(LoadBalancer)** 또는 **Ingress**를 함께 사용합니다. Ingress는 여러 Service를 하나의 도메인과 URL 경로로 관리하는 역할을 하므로, 운영 환경에서는 **LoadBalancer + Ingress Controller** 구성이 매우 일반적입니다.

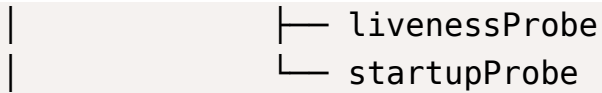
※ Envoy : 어플리케이션 앞에서 네트워크 요청을 대신 처리해주는 고성능 프록시 서버임.

- Deployment configuration

```

Deployment
├── replicas
├── strategy
├── selector
├── template
│   ├── labels
│   └── containers
│       ├── image
│       ├── env
│       ├── resources
│       ├── volumeMounts
│       └── readinessProbe

```



각 항목 관련 사항

- selector
최초 Deployment에서만 만들 수 있음. 이후 수정 시 label 추가는 template의 labels에 작성 필요
- strategy

- RollingUpdate

```

strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 1

```

maxSurge와 maxUnavailable은 %로 표현도 가능하며, replicaSet의 pod 수 기준 rollingUpdate 과정에서 추가로 생성이 가능한 pod 수와 사용 불가능한 (Ready가 아니거나 삭제된) pod 수를 제어함. 예를들어, 각각이 1이고 ReplicaSet이 3이면 총 4개의 pod가 있을 수 있으며, 최소 2개의 Pod는 항상 Ready 상태여야 함을 의미함.

- paused

```

apiVersion: apps/v1
kind: Deployment

metadata:
  name: my-app

spec:
  paused: true

  replicas: 3

```

```

selector:
  matchLabels:
    app: my-app

template:
  metadata:
    labels:
      app: my-app

spec:
  containers:
  - name: app
    image: myapp:v2

```

- Paused 가 True면 새 이미지를 적어도 ReplicaSet이 새로 생성되지 않음.
여러 변경 사항을 나눠서 적용해야 하는 경우, Paused: True가 설정되어 있지 않으면 배포도 update도 여러 번 발생함. 운영 중 변경을 여러 단계로 나눠서 적용하거나 roll-out을 지연시켜야 하는 특수 상황에서 사용함.
→ 실무에서는 주로 다음 명령어를 사용하여 pause 시키고, 재개함.

```

kubectl rollout pause deployment my-app
kubectl rollout resume deployment my-app

```

- ReadinessProbe & LivenessProbe

Probe	질문	실패 시 동작
readinessProbe	"이 Pod가 요청을 처리할 준비가 되었나?"	Service가 이 Pod로 트래픽을 보내지 않음 (컨테이너는 계속 실행)
livenessProbe	"이 컨테이너가 정상적으로 살아 있나?"	컨테이너를 재시작함

- 배포 전략
 - Canary 배포는 다양한 방식으로 설정이 가능하나, 가장 단순한 방식은 두 Deployment를 작성하고, 공통 서비스를 연결함. 이때 replicaset의 수를 조정하여 비율을 맞춤.

전략	동작	무중단	위험도	Kubernetes 구현
Recreate	기존 Pod를 모두 종료한 뒤 신규 Pod 생성	불가능	높음	Deployment 하나
Rolling Update	기존 Pod를 조금씩 신규 Pod로 교체	가능	보통	Deployment 하나
Blue-Green	구버전과 신버전을 동시에 운영 후 Service 전환	가능	낮음	Deployment 2개 + Service
Canary	일부 사용자/트래픽만 신버전으로 전달	가능	매우 낮음	Deployment 2개 + 라우팅
A/B Testing	사용자 조건에 따라 서로 다른 버전 제공	가능	낮음	Deployment 2개 + 조건 기반 라우팅
Shadow	실제 트래픽을 복제해 신버전에 전달하되 응답은 버림	가능	매우 낮음	Gateway/Service Mesh 필요